AIN SHAMS UNIVERSITY
FACULTY OF ENGINEERING

CPU Scheduler Project

ChronoCore

Operating Systems (CSE335s)

Submitted to: Dr. Saher

Team Members

Name	ID
Youssef Yacoub	2200649
Mohamed Khaled Ahmed	2201057
Ahmed Nagger Hassan	2200306
Mosa Abdulaziz	2200257
Bassam Hussam	2200084
Omar Mohamed Mahmoud Eid	2200284

Table of Contents

1 Introduction 1

2 System Architecture 1

3 Models (src/Models) 2

4 UI (src/UI) 5

5 Controllers (src/Controllers) 8

6 Scheduling Algorithms (The Engines) 10

7 The Live Simulation State Machine 11

8 Testing 12

9 Technologies Used 12

10 Project Resources 13

1 Introduction

ChronoCore is a desktop CPU-scheduling simulator built to make the internals of common operating-system scheduling algorithms visible and interactive. Instead of presenting a Gantt chart as a static diagram, ChronoCore lets the user watch the ready queue, CPU load, and timeline evolve in real time while processes arrive, run, and complete.

The application focuses on four classical scheduling strategies that every operating-systems course covers:

- **First-Come First-Served (FCFS)** — non-preemptive, arrival-ordered.
- **Shortest Job First (SJF)** — both non-preemptive and preemptive (Shortest Remaining Time First) variants.
- **Priority Scheduling** — both non-preemptive and preemptive, with *smaller priority number means higher priority*, and FCFS used as the deterministic tie-breaker.
- **Round Robin (RR)** — with a user-chosen quantum.

Two simulation modes

ChronoCore supports two execution paradigms:

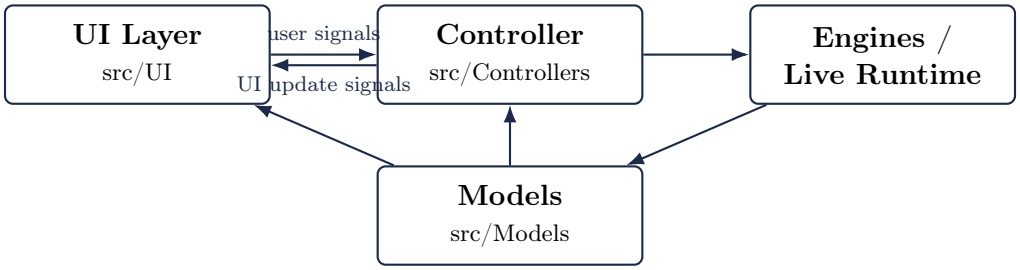
Static Mode
: The full schedule is computed instantly from the declared process set. A single call to an algorithm engine yields the entire timeline and final metrics (average waiting time, turnaround, response time). Ideal for verifying homework answers.

Live Mode
: A QTimer drives the simulation at **one simulation unit per real-time second**. Processes tick forward visibly; new processes can be added *while the scheduler is running*, and the UI updates the active-state table, Gantt chart, and metric cards on every tick.

This split between a pure static engine and an event-driven live runtime is the backbone of the project and shapes every design choice that follows.

2 System Architecture

ChronoCore follows a strict **three-layer separation of concerns**: the views have no knowledge of scheduling math, the controller has no knowledge of widgets, and the scheduling engines have no knowledge of either. They communicate only through well-typed data objects and Qt signals.



Why this layout?

- The UI only renders what it is told; any view technology (Qt today, web tomorrow) can be swapped without touching the scheduler.
- The controller is the only layer aware of Qt outside the UI.

- Scheduling engines are pure Python with no I/O, making them trivially unit-testable (see `tests/test_engines.py`).

3 Models (src/Models)

The Models package defines the *shared vocabulary* that every other layer speaks. There is no behavior here beyond simple helpers — only data structures and enums.

Why immutable-friendly data?

ChronoCore uses Python `@dataclass(slots=True)` for every model. This gives three concrete benefits:

1. **No UI race conditions.** When the controller emits a `SchedulerSnapshot`, the UI receives a finished, consistent object. The live runtime cannot mutate it after the fact because Qt signals deliver the object by reference *and* the controller never mutates snapshots after emission.
2. **Memory efficiency.** `slots=True` prevents a per-instance `__dict__`, important when the live runtime creates hundreds of `ProcessRuntimeState` entries per simulation.
3. **Clear contracts.** Every field is typed; future contributors can read a 20-line file and know exactly what flows through the system.

The core types

- **ProcessStatus** — enum of lifecycle states used in the monitor table (`READY`, `RUNNING`, `WAITING`, `FINISHED`).
- **ProcessSpec** — the user-declared process (pid, arrival, burst, optional priority, insertion order). Treated as immutable by convention.
- **ProcessRuntimeState** — the per-process mutable runtime view used *inside* the engines: remaining time, first-start timestamp, completion timestamp, current status.
- **AlgorithmKey / SimulationMode** — string-valued enums shared by the controller, engines, and live runtime so the UI can pass plain strings while the backend works with typed values.
- **TimelineSlice** — a single Gantt segment (pid, start, end).
- **ProcessMetrics** — final per-process metrics (waiting, turnaround, response).
- **ScheduleResult** — the full output of a static run (timeline + per-process metrics + averages).
- **SchedulerSnapshot** — one live tick of state broadcast to the UI: current time, running pid, table rows, CPU load, progress, averages.

Lifecycle enum and immutable input

```python
class ProcessStatus(str, Enum):
    READY = "READY"
    RUNNING = "RUNNING"
    WAITING = "WAITING"
    FINISHED = "FINISHED"


```

```python
@dataclass(slots=True)
class ProcessSpec:
    """Immutable input process definition provided by the UI."""
    pid: str
    arrival_time: float
    burst_time: float
    priority: Optional[int] = None
    insertion_order: int = 0

    def to_dict(self) -> dict:
        return {
            "pid": self.pid,
            "arrival": self.arrival_time,
            "burst": self.burst_time,
            "priority": self.priority,
        }
```

Listing 1: `src/Models/process.py` — status enum and the input spec.

The `insertion_order` field deserves a note: it is the tie-breaker used everywhere a scheduling key can collide (two equal bursts in SJF, two equal priorities in Priority). Storing it on the spec rather than recomputing it keeps engines deterministic and keeps equality of schedules reproducible.

Mutable runtime view

```python
@dataclass(slots=True)
class ProcessRuntimeState:
    """Mutable runtime state used during simulation."""
    spec: ProcessSpec
    remaining_time: float = field(init=False)
    first_start_time: Optional[float] = None
    completion_time: Optional[float] = None
    status: ProcessStatus = ProcessStatus.READY

    def __post_init__(self) -> None:
        self.remaining_time = self.spec.burst_time

    @property
    def pid(self) -> str: return self.spec.pid
    @property
    def arrival_time(self) -> float: return self.spec.arrival_time
    @property
    def burst_time(self) -> float: return self.spec.burst_time
    @property
    def priority(self) -> Optional[int]: return self.spec.priority
    @property
    def is_complete(self) -> bool: return self.remaining_time <= 0

    def to_table_row(self, current_time: float = 0.0) -> dict:
        active_end = (self.completion_time
                      if self.completion_time is not None
                      else current_time)
        turnaround = (max(0.0, active_end - self.arrival_time)
                      if self.is_complete else None)
        waiting = (max(0.0, turnaround - self.burst_time)
                   if turnaround is not None else None)
        return {
            "pid": self.pid,
            "arrival": round(self.arrival_time, 2),
            "burst": round(self.burst_time, 2),
```

```
            "remaining": max(0.0, round(self.remaining_time, 2)),
            "priority": self.priority,
            "status": self.status.value,
            "waiting_time": (round(waiting, 2)
                             if waiting is not None else None),
            "turnaround_time": (round(turnaround, 2)
                                if turnaround is not None else None),
        }
```

Listing 2: `src/Models/process.py` — ProcessRuntimeState.

Algorithm and mode enums

```
class AlgorithmKey(str, Enum):
    FCFS = "fcfs"
    SJF = "sjf"
    PRIORITY = "priority"
    ROUND_ROBIN = "round_robin"


class SimulationMode(str, Enum):
    STATIC = "static"
    LIVE = "live"
```

Listing 3: `src/Models/schedule_types.py` — enums.

Timeline and metrics

```
@dataclass(slots=True)
class TimelineSlice:
    """Represents one contiguous gantt segment."""
    pid: str
    start: float
    end: float
    is_context_switch: bool = False

    def to_dict(self) -> dict:
        return {
            "pid": self.pid,
            "start": self.start,
            "end": self.end,
            "is_context_switch": self.is_context_switch,
        }


@dataclass(slots=True)
class ProcessMetrics:
    pid: str
    waiting_time: float
    turnaround_time: float
    response_time: float

    def to_dict(self) -> dict:
        return {
            "pid": self.pid,
            "waiting_time": self.waiting_time,
            "turnaround_time": self.turnaround_time,
            "response_time": self.response_time,
        }
```

Listing 4: `src/Models/schedule_types.py` — slice and metrics.

Static result and live snapshot

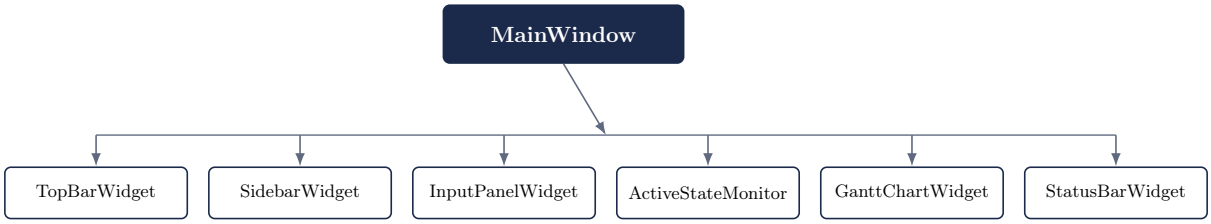
```python
@dataclass(slots=True)
class ScheduleResult:
    timeline: list[TimelineSlice]
    process_metrics: list[ProcessMetrics]
    average_wait_time: float
    average_turnaround_time: float

    def timeline_payload(self) -> list[dict]:
        return [item.to_dict() for item in self.timeline]


@dataclass(slots=True)
class SchedulerSnapshot:
    """One live-tick snapshot for UI updates."""
    current_time: float
    running_pid: Optional[str]
    table_rows: list[dict]
    cpu_load: int
    total_processes: int
    progress: int
    new_slice: Optional[TimelineSlice] = None
    completed: bool = False
    average_wait: float = 0.0
    average_turnaround: float = 0.0
    delta_wait: float = 0.0
    delta_turnaround: float = 0.0
```

Listing 5: `src/Models/schedule_types.py` — result and snapshot.

4 UI (src/UI)

The UI layer is built on **PySide6**, the official Python binding for Qt. Each major region of the window is a small custom `QWidget` subclass with a single responsibility. `MainWindow` composes them; it never reaches into their internal state. Communication is purely through Qt's **signal/slot** mechanism.

Widget architecture



Per-widget signals and slots

Each custom widget exposes a small, documented set of Qt **signals** (outbound events) and public **slot-like methods** (inbound updates). The controller is the only listener on these signals, and

the only caller of these methods. The sections below walk through every signal, what triggers it, and what the controller does in response.

TopBarWidget

Role: title bar, Live/Static toggle, Run/Stop buttons, and the global progress bar.

Defined outbound signals (emitted to the controller):

- `simulation_mode_changed(str)` — **Trigger:** user clicks the Live/Static toggle button. **Effect:** the controller calls `set_mode(mode)` which updates the internal `SimulationMode`. The next *Run* click will choose the static engine or the live runtime accordingly.
- `run_simulation_clicked()` — **Trigger:** user presses the *Run* button. **Effect:** the controller validates that at least one process exists, emits `status_changed("RUNNING")`, then either computes the entire static schedule or builds a fresh `LiveRuntimeState` and starts the 1-second `QTimer`.
- `stop_simulation_clicked()` — **Trigger:** user presses the *Stop* button. **Effect:** the controller stops the `QTimer`, emits `status_changed("READY")`, and drops the CPU-load indicator back to 0%.

Inbound slot (called by the controller): `set_progress(int)` — updates the progress bar from 0 to 100 on every tick (or once after a static run).

SidebarWidget

Role: algorithm selector (FCFS, SJF, Priority, Round Robin), preemptive checkbox, and the *New Scenario* reset button.

Defined outbound signals (emitted to the controller):

- `algorithm_selected(str)` — **Trigger:** user clicks one of the four algorithm cards. **Effect:** the controller calls `set_algorithm` which converts the string into an `AlgorithmKey` enum and (for Round Robin) re-emits the quantum lock state so the input panel shows/hides the quantum field correctly.
- `preemptive_toggled(bool)` — **Trigger:** user checks the *Preemptive* checkbox (only visible for SJF and Priority). **Effect:** the controller stores the flag in `EngineConfig`; static runs and live runs both read from it when deciding whether to preempt on new arrivals.
- `new_scenario_clicked()` — **Trigger:** user presses the *New Scenario* button. **Effect:** the controller calls `reset_scenario`, which stops any running simulation, clears the process list, resets counters, and emits cleared payloads to every display widget.

InputPanelWidget

Role: the process-entry form and the averaged-results readout.

Defined outbound signals (emitted to the controller):

- `process_added(dict)` — **Trigger:** user fills in the form and clicks *Add Process*. The dict carries `pid, arrival, burst, priority, quantum` (the latter two optional). **Effect:** the controller builds a `ProcessSpec`, stamps it with the next `insertion_order`, appends it to the master list, emits `total_processes_changed`, and — if a live simulation is already running — injects the spec into the active `LiveRuntimeState` via `add_process_live`.

- `input_fields_cleared()` — **Trigger:** emitted immediately after a successful add, signalling that the form has been auto-reset. **Effect:** currently informational; useful for tests and for possible future logging.

Inbound slots (called by the controller):

- `update_results(avg_wait, avg_tat, ...)` — called once the static run finishes or on every live tick, to refresh the two metric cards.
- `set_status_badge(str)` — switches the small header badge between READY, RUNNING, and FINISHED.
- `set_quantum_locked(bool)` — disables the quantum field after a Round-Robin simulation starts (the quantum cannot change mid-run by design).
- `set_algorithm_mode(algo, preemptive)` — shows/hides the priority and quantum fields depending on the active algorithm.

ActiveStateMonitor

Role: real-time process table with eight columns (Process, Arrival, Burst, Remaining, Priority, Status, Wait, Turnaround) plus a CPU-load bar.

Outbound signals: none — this widget is display-only. Inbound slots from the controller are:

- `update_process_table(list[dict])` — receives a list of row dicts. Each dict comes from `ProcessRuntimeState.to_table_row` and is rendered into the table in a single repaint.
- `set_total_processes(int)` — updates the header count.
- `set_cpu_load(int)` — updates the CPU-load progress bar; the controller emits 100% while a process is running and 0% while idle.

GanttChartWidget

Role: scrollable timeline drawn with a custom `QPainter`.

Outbound signals: none — display-only. Inbound slots from the controller are:

- `add_gantt_block(pid, start, end, is_context_switch)` — appends a single timeline segment. In static mode the controller emits one call per `TimelineSlice` in the computed result; in live mode it emits exactly one call per tick (whenever `snap.new_slice` is not `None`).
- `clear_chart()` — wipes the timeline, called on reset.
- `set_time_range(float)` — adjusts the horizontal scale if the user declares a long scenario up front.

StatusBarWidget

Role: bottom bar with a system-health dot, the active-algorithm label, and a live uptime clock.

Outbound signals: none — display-only. Inbound slots from the controller are:

- `set_system_status(bool)` — flips the health dot between green (healthy, a simulation is running) and grey (idle).
- `set_active_algorithm(str)` — updates the label with the currently selected algorithm's display name.

Event-driven communication

Every piece of user input is a Qt **signal** emitted by a widget; every display update is a Qt **slot** on a widget called by the controller through a connected signal. Views never call scheduling code; the controller never calls `repaint()` directly. This decoupling is what allows the entire scheduling engine layer to be tested with plain `pytest`, with no Qt event loop running.

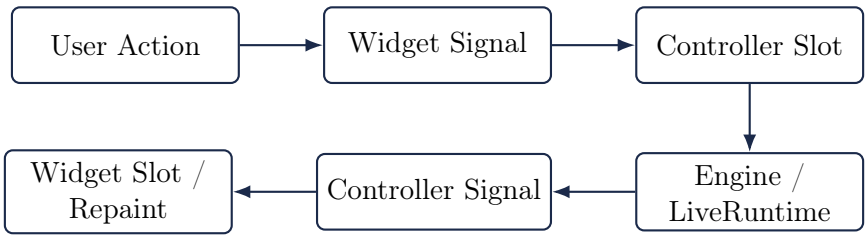
5 Controllers (src/Controllers)

`SchedulerController` is a `QObject` — the only object outside the UI package that imports Qt. Conceptually it is a **bridge** between two worlds: the user-facing widgets that emit human actions, and the pure-Python scheduling engines that compute timelines and metrics.

Bridge responsibilities

1. **Receive** user actions (clicks, form submissions, toggles) via slots connected to widget signals.
2. **Translate** UI payloads (plain `dict`) into typed domain objects (`ProcessSpec`, `AlgorithmKey`, `SimulationMode`).
3. **Orchestrate** the right execution path: call the static engine via `EngineFactory.create(algo)` and its `compute_static(...)` method for static mode, or construct a `LiveRuntimeState` and drive it with a `QTimer` for live mode.
4. **Emit** Qt signals that carry the resulting data back to the UI (`gantt_block_added`, `process_table_updated`, `results_updated`, and more).

Dataflow



Controller-emitted signals

The controller defines nine outbound signals. Each one fires in specific situations and is consumed by exactly one UI slot (wired in `bind_window`). This is the complete contract.

- `progress_updated(int)` — **Fires:** every tick in live mode, once at the end of a static run. **Payload:** 0–100 percentage of overall simulation progress. **Consumer:** `TopBarWidget.set_progress` (updates the header progress bar).
- `process_table_updated(list)` — **Fires:** every tick, on reset, and once after a static run completes. **Payload:** a list of row dicts produced by `ProcessRuntimeState.to_table_row`. **Consumer:** `ActiveStateMonitor.update_process_table` (redraws the table atomically).
- `total_processes_changed(int)` — **Fires:** whenever the master process list grows (via `add_process`) or shrinks (via `reset_scenario`). **Consumer:** `ActiveStateMonitor.set_total_processes` (updates the count chip in the table header).

- `cpu_load_changed(int)` — **Fires:** on every tick (100 while a process runs, 0 while idle) and on stop/reset (0). **Consumer:** `ActiveStateMonitor.set_cpu_load` (animates the CPU-load bar).
- `gantt_block_added(pid, start, end, is_cs)` — **Fires:** once per `TimelineSlice` in static mode; once per live tick that produced a `new_slice`. **Consumer:** `GanttChartWidget.add_gantt_block` (paints one segment).
- `results_updated(avg_wait, avg_tat, dw, dt)` — **Fires:** every live tick and once at the end of a static run. **Consumer:** `InputPanelWidget.update_results` (refreshes the two metric cards).
- `status_changed(str)` — **Fires:** on `start_simulation` (RUNNING), on `stop_simulation` / `reset_scenario` (READY), and when a simulation naturally completes (FINISHED). **Consumer:** `InputPanelWidget.set_status_badge` (colours the header badge).
- `system_health_changed(bool)` — **Fires:** `True` when the user starts a simulation, `False` on stop / reset / completion. **Consumer:** `StatusBarWidget.set_system_status` (green vs grey dot in the status bar).
- `quantum_lock_changed(bool)` — **Fires:** `True` once the first Round-Robin process is added (to fix the quantum for the run), `False` when the user presses *New Scenario*. **Consumer:** `InputPanelWidget.set_quantum_locked` (disables/enables the quantum input field).

Controller slots (inbound from the UI)

- `set_mode(str)` — called on every click of the Live/Static toggle.
- `set_algorithm(str)` — called when the user picks an algorithm card in the sidebar.
- `set_preemptive(bool)` — called when the Preemptive checkbox changes state.
- `add_process(dict)` — called when the user submits the form; wraps the payload in a `ProcessSpec` and, if a live simulation is already ticking, injects it via `add_process_live`.
- `start_simulation()` — runs either the static engine or starts the `QTimer` for the live runtime.
- `stop_simulation()` — stops the timer and emits the "ready" status and zero CPU load.
- `reset_scenario()` — clears everything and broadcasts empty payloads to every display widget.
- `_on_live_tick()` — private slot connected to the `QTimer`; calls `runtime.tick()` and translates the resulting snapshot into six outbound signal emissions.

A single wiring point: `bind_window`

Every UI-controller connection lives in one method, so a new developer can read the entire contract in a single screen.

```
def bind_window(self, window) -> None:
    # UI -> Controller (user actions)
    tb = window.top_bar
    sb = window.sidebar
    ip = window.input_panel
    tb.run_simulation_clicked.connect(self.start_simulation)
    tb.stop_simulation_clicked.connect(self.stop_simulation)
    tb.simulation_mode_changed.connect(self.set_mode)
    sb.algorithm_selected.connect(self.set_algorithm)
    sb.preemptive_toggled.connect(self.set_preemptive)
```

```
    ip.process_added.connect(self.add_process)
    sb.new_scenario_clicked.connect(self.reset_scenario)

    # Controller -> UI (data updates)
    am = window.active_monitor
    self.progress_updated.connect(tb.set_progress)
    self.process_table_updated.connect(am.update_process_table)
    self.total_processes_changed.connect(am.set_total_processes)
    self.cpu_load_changed.connect(am.set_cpu_load)
    self.gantt_block_added.connect(window.gantt_chart.add_gantt_block)
    self.results_updated.connect(ip.update_results)
    self.status_changed.connect(ip.set_status_badge)
    self.system_health_changed.connect(window.status_bar.set_system_status)
    self.quantum_lock_changed.connect(ip.set_quantum_locked)
```

Listing 6: `src/Controllers/scheduler_controller.py`.

When the user clicks *Add Process*, the signal carries a dict into the controller's `add_process` slot. The slot builds a `ProcessSpec`, appends it to the authoritative process list, and — if a live simulation is running — hands it to the runtime via `add_process_live(spec)` for immediate ingestion on the next tick.

6 Scheduling Algorithms (The Engines)

All four scheduling algorithms live in `src/Controllers/engines/`. Each engine is a pure Python class with a single entry point, `compute_static(processes, config) -> ScheduleResult`, that returns the complete timeline and per-process metrics. Engines share no state; they receive inputs and produce outputs.

Factory Design Pattern

To keep the controller code oblivious to which algorithm is active, engines are constructed through an `EngineFactory`. The factory is the only piece of code that maps `AlgorithmKey` values to concrete engine classes. Adding a new algorithm (say, Multilevel Feedback Queue) is a three-step change: add a key to the enum, add the engine class, register it in the factory. The controller code does not change at all.

```python
class EngineFactory:
    @staticmethod
    def create(algorithm: str) -> BaseSchedulerEngine:
        key = AlgorithmKey(algorithm)
        if key is AlgorithmKey.FCFS:
            return FCFSEngine()
        if key is AlgorithmKey.SJF:
            return SJFEngine()
        if key is AlgorithmKey.PRIORITY:
            return PriorityEngine()
        if key is AlgorithmKey.ROUND_ROBIN:
            return RoundRobinEngine()
        raise ValueError(f"Unsupported algorithm: {algorithm}")
```

Listing 7: `src/Controllers/engines/factory.py`.

Per-engine notes

- **FCFSEngine.** Sorts by `(arrival_time, insertion_order)`, then runs each process to completion. Deterministic and the simplest of the four.

- **SJFEngine.** Accepts `config.preemptive`. In non-preemptive mode it picks the shortest burst among arrived processes; in preemptive mode (SRTF) it recomputes the pick every time a new process arrives.
- **PriorityEngine.** Smaller priority number wins. **Ties fall back to FCFS** via the sort key `(priority, arrival_time, insertion_order)`. This rule is intentional and documented: it keeps the schedule predictable when two processes share a priority.
- **RoundRobinEngine.** Consumes a `quantum` from the config and maintains a FIFO queue of ready processes. A process that exhausts its quantum is appended to the back of the queue.

Shared helpers via `BaseSchedulerEngine`

Every engine inherits from `BaseSchedulerEngine`, which provides reusable primitives: building `ProcessRuntimeState` lists, timeline-slice merging (so consecutive same-pid segments collapse into one Gantt block), and the final metrics computation (waiting, turnaround, response, and averages). This keeps each concrete engine focused on *which* process to pick next and delegates the bookkeeping to the base.

7 The Live Simulation State Machine

The file `src/Controllers/live_runtime.py` implements the heart of live mode: a state machine that answers a single question every second: *"which process runs for the next one unit of time, and what changes as a result?"*

The artificial tick clock

The controller owns a `QTimer` with a 1000ms interval. On every timeout it calls `runtime.tick()`, which:

1. Ingests any processes whose `arrival_time` is now reached.
2. Asks the scheduling policy which process should run this unit.
3. Advances the simulation clock by one unit.
4. Appends a `TimelineSlice` to the Gantt timeline.
5. Either marks the process finished or puts it back in the waiting room.
6. Returns a `SchedulerSnapshot` for the controller to broadcast.

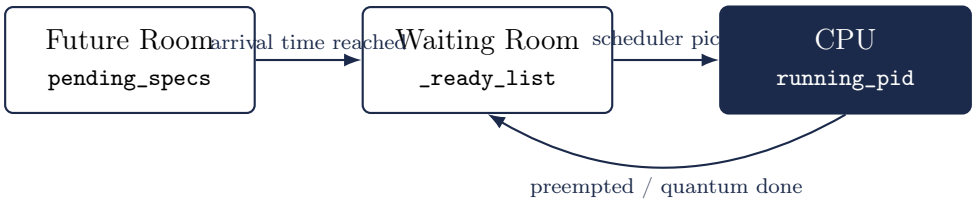
This is deliberate: one real-time second equals one simulation unit, so the user can *see* the scheduler making decisions and can add new processes mid-run without ever pausing the simulation.

"Future Room" vs "Waiting Room"

Internally the runtime maintains two buckets:

`pending_specs` **(the Future Room)**
: Processes declared by the user whose `arrival_time` is still in the future. They are not in the scheduler's view yet. Each tick the runtime checks the front of this list; once a process's arrival time is reached, it graduates into the waiting room.

`_ready_list` / `_rr_queue` (the Waiting Room)
Live processes that have arrived but are not currently on the CPU. Non-RR algorithms use a plain list that the scheduler resorts as needed; Round Robin uses a `deque` to preserve FIFO order.



Dynamic process insertion

Because the Future Room is just a sorted list, adding a process mid-run is trivial. `SchedulerController.add_process` calls `runtime.add_process_live(spec)`, which inserts the new spec into `pending_specs` sorted by `(arrival_time, insertion_order)`. The very next tick picks it up through `_ingest_arrivals`.

8 Testing

Because the scheduling engines are pure Python with no Qt dependencies, they are tested with standard `pytest`. The file `tests/test_engines.py` covers the deterministic properties that matter most:

- FCFS preserves arrival order.
- SJF non-preemptive orders by burst.
- SJF preemptive (SRTF) preempts on shorter arrivals.
- Priority uses FCFS as its tie-breaker.
- Round Robin respects the quantum across a full cycle.

Tests are run from the project root with:

```
python -m pytest -q tests/
```

Listing 8: Running the test suite.

A companion test, `tests/test_controller_static.py`, spins up an offscreen Qt application and validates that the full controller-to-UI flow still emits the expected payloads in static mode.

9 Technologies Used

- **Python 3.11+** — modern dataclass features (`slots=True`), structural pattern support, and precise type hints.
- **PySide6 (Qt for Python)** — cross-platform widgets, signals, slots, custom painting (QPainter), and QTimer for the live clock.
- **Python dataclasses** — lightweight, immutable-friendly model objects that form the shared vocabulary across layers.
- **pytest** — fast, zero-configuration unit-test runner for the scheduling engines.

- **PyInstaller** — optional packaging into a single desktop executable for distribution.

10 Project Resources

Full source code, assets, and build artifacts for ChronoCore are hosted on Google Drive:

Google Drive Repository

https://drive.google.com/drive/folders/1ZbIezayaMmvBtG_Iw7JgcIDYCkv1R367?usp=sharing

End of report.